

Relatório de Atividades: Hacking de Binários para Tradução de Jogos

Isabelle Oliveira Bicudo e Kaê de Oliveira Budke

¹Universidade Federal de Mato Grosso do Sul (UFMS)
CEP 79.070-900 – Campo Grande – MS – Brasil

isabelle.bicudo@ufms.br, kae.budke@ufms.br

1. Introdução

Este relatório detalha o processo de modificação de uma ROM do jogo *Super Mario World* de Super Nintendo com o objetivo de traduzir os textos do inglês para o português. A técnica central utilizada foi o *ROM Hacking*, que consiste na edição direta do arquivo binário do jogo por meio de ferramentas de edição hexadecimal e scripts customizados.

O projeto aborda desafios comuns na modificação de binários, como a necessidade de mapear tabelas de caracteres customizadas, onde os valores hexadecimais não correspondem diretamente ao padrão ASCII. Foram desenvolvidas três ferramentas em Python para automatizar e facilitar as tarefas de encontrar, extrair e reinserir os textos na ROM.

Os textos-alvo escolhidos para a tradução foram a mensagem de introdução do jogo e o diálogo exibido durante o resgate do personagem *Yoshi*.

Mensagem de introdução:

“Welcome! This is Dinosaur Land. In this strange land we find that Princess Toadstool is missing again! Looks like Bowser is at it again!”

Diálogo de resgate do Yoshi:

“Hooray! Thank you for rescuing me. My name is Yoshi. On my way to rescue my friends, Bowser trapped me in that egg.”

2. Metodologia

O fluxo de trabalho foi estruturado em etapas sequenciais, utilizando um conjunto de ferramentas desenvolvidas especificamente para este projeto, além de software de análise hexadecimal.

2.1. Análise Inicial e Criação da Tabela de Caracteres

A primeira etapa consistiu na análise da ROM `SMario.sfc` utilizando o editor hexadecimal **Translhexction16c**, com o objetivo de identificar a forma como os textos eram armazenados. Verificou-se que o jogo emprega uma tabela de caracteres própria. A partir da ferramenta de busca relativa do **Translhexction16c**, foi possível localizar correspondências entre sequências de bytes e palavras conhecidas (como “*strange*”), o que permitiu a elaboração de uma tabela de mapeamento inicial (`mario_dialogo_raw.tbl`). Posteriormente, essa tabela foi refinada manualmente (`mario_dialogo_hum.tbl`), de modo a incluir outros caracteres necessários para a tradução.

2.2. Ferramentas Desenvolvidas

Para automatizar o processo, foram criados três scripts em Python:

- `finder.py`: ferramenta de busca que recebe uma *string* e uma tabela de caracteres (`.tbl`) como entrada. Ela codifica o texto para sua representação hexadecimal correspondente e localiza sua posição (*offset*) dentro da ROM.
- `extractor.py`: script responsável por extrair os textos de regiões predefinidas da ROM. Ele lê um arquivo `ranges.json`, que especifica os endereços de início e fim dos blocos de texto, e utiliza a tabela de caracteres para decodificar os bytes, salvando o resultado em um arquivo CSV (`saida.csv`).
- `injector.py`: ferramenta final do processo, que lê o arquivo CSV com os textos já traduzidos (`traduzido.csv`), codifica as novas *strings* para hexadecimal usando a tabela e as insere de volta na ROM nos *offsets* originais. O script garante que o novo texto não ultrapasse o espaço alocado, podendo truncar ou preencher quando necessário.

2.3. Fluxo de Execução

O processo completo seguiu os seguintes passos:

1. **Geração da Tabela:** análise da ROM no **Translhexction16c** para gerar a tabela de caracteres base (`mario_dialogo_raw.tbl`).
2. **Localização dos Textos:** uso do script `finder.py` para encontrar os *offsets* dos textos-alvo.
3. **Definição das Regiões:** mapeamento das regiões de memória no arquivo `ranges.json`.
4. **Extração:** execução do `extractor.py` para exportar os textos em um arquivo CSV.
5. **Tradução:** edição manual do CSV, traduzindo a coluna de texto para o português sem exceder o limite original.
6. **Injeção:** reinserção dos textos com `injector.py`, gerando uma nova ROM (`ROM_BR_NB.sfc`).
7. **Teste:** validação em emuladores (**SNES9x** e **OpenEmu**).

3. Resultados

A tradução dos textos não foi um processo instantâneo. As imagens a seguir documentam a evolução da injeção de texto na ROM, desde as primeiras tentativas com caracteres incorretos até a versão final e corrigida.

3.1. Conjunto de Teste

As primeiras tentativas resultaram em caracteres quebrados ou mal interpretados, indicando falhas no mapeamento inicial.



Figura 1. Tentativa inicial de injeção de texto.

Um pequeno avanço ocorreu quando alguns caracteres passaram a ser exibidos corretamente.



Figura 2. Correção parcial de caracteres.

Outras etapas intermediárias mostraram ajustes graduais até a obtenção de uma versão quase estável, com erros pontuais de mapeamento.



Figura 3. Versão quase final da tradução.

Finalmente, a tradução foi concluída com sucesso, preservando a integridade da ROM e mantendo a jogabilidade intacta.

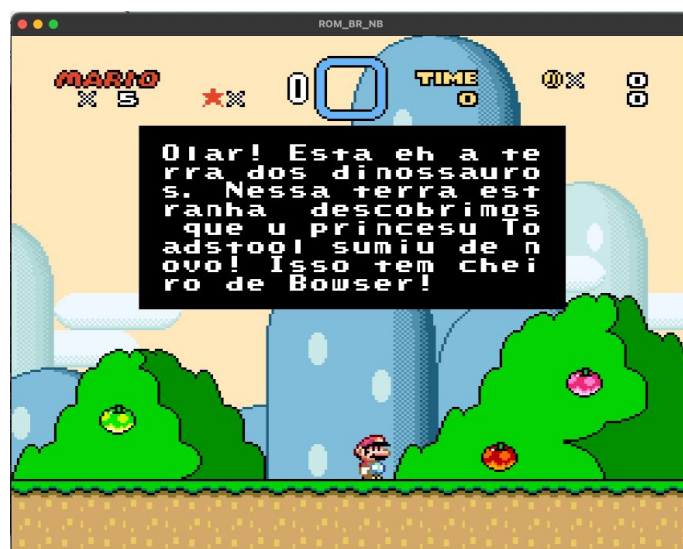


Figura 4. Tradução final validada nos emuladores.

4. Conclusão

O trabalho demonstrou com sucesso a viabilidade da técnica de *binary hacking* aplicada à modificação e tradução de jogos eletrônicos. A principal dificuldade encontrada foi a interpretação dos dados em formato hexadecimal sem um mapeamento de caracteres conhecido, obstáculo que foi superado com a criação de uma tabela customizada.

O desenvolvimento de scripts específicos (`finder.py`, `extractor.py`, `injector.py`) foi fundamental para otimizar o processo, tornando-o mais rápido, preciso e escalável para a tradução de outros textos dentro do mesmo jogo. O projeto cumpriu

todos os seus objetivos, resultando em uma ROM funcional com os textos selecionados traduzidos para o português.